

Name: Devarjya
Reg No: 240968040

1) Write a MPI program to read a value M and NxM elements in the root process. Root process sends M elements to each process. Each process finds average of M elements it received and sends these average values to root. Root collects all the values and finds the total average. Use collective communication routines. Use N number of processes.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size, M = 0;
    double *matrix = NULL, *local_elements = NULL, *all_avgs = NULL;
    double local_sum = 0.0, local_avg = 0.0, total_avg = 0.0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Name: Devarjya, Reg No: 240968040\n");
    }

    int N = size;

    if (rank == 0) {
        printf("Enter value for M: ");
        fflush(stdout);
        scanf("%d", &M);
    }

    MPI_Bcast(&M, 1, MPI_INT, 0, MPI_COMM_WORLD);

    local_elements = (double*)malloc(M * sizeof(double));

    if (rank == 0) {
        matrix = (double*)malloc(N * M * sizeof(double));
        printf("Enter %d elements (%d rows x %d columns):\n", N * M, N, M);
        fflush(stdout);
        for (int i = 0; i < N * M; i++) {
            scanf("%lf", &matrix[i]);
        }
        all_avgs = (double*)malloc(N * sizeof(double));
    }

    MPI_Scatter(matrix, M, MPI_DOUBLE, local_elements, M, MPI_DOUBLE, 0,
MPI_COMM_WORLD);
```

```

for (int i = 0; i < M; i++) {
    local_sum += local_elements[i];
}
local_avg = local_sum / M;

MPI_Gather(&local_avg, 1, MPI_DOUBLE, all_avgs, 1, MPI_DOUBLE, 0,
MPI_COMM_WORLD);

if (rank == 0) {
    double sum_avgs = 0.0;
    for (int i = 0; i < N; i++) {
        sum_avgs += all_avgs[i];
    }
    total_avg = sum_avgs / N;
    printf("Total average of all elements: %.4f\n", total_avg);

    free(matrix);
    free(all_avgs);
}

free(local_elements);
MPI_Finalize();
return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 6$ mpicc q1.c -o q1
pcap1@PC-36:~/Documents/240968040/week 6$ mpirun -n 3 q1
Name: Devarjya, Reg No: 240968040
Enter value for M: 3
Enter 9 elements (3 rows x 3 columns):
1
2
3
4
5
6
7
8
9
Total average of all elements: 5.0000

```

2) Write a MPI Program to read two strings S1 and S2 of same length in the root process. Using N process including the root (string length is evenly divisible by N), produce the resultant string as shown below. Display the resultant string in the root process. Write the program using Collective communication routines.

Example:

String S1: string String S2: length Resultant String : slternigtgh

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size, length = 0;
    char *S1 = NULL, *S2 = NULL, *resultant = NULL;
    char *local_s1 = NULL, *local_s2 = NULL, *local_res = NULL;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Name: Devarjya, Reg No: 240968040\n");
    }

    if (rank == 0) {
        S1 = (char*)malloc(256 * sizeof(char));
        S2 = (char*)malloc(256 * sizeof(char));

        printf("Enter string S1: ");
        fflush(stdout);
        scanf("%s", S1);

        printf("Enter string S2: ");
        fflush(stdout);
        scanf("%s", S2);

        length = strlen(S1);
    }

    MPI_Bcast(&length, 1, MPI_INT, 0, MPI_COMM_WORLD);

    int chunk_size = length / size;

    local_s1 = (char*)malloc((chunk_size + 1) * sizeof(char));
    local_s2 = (char*)malloc((chunk_size + 1) * sizeof(char));
    local_res = (char*)malloc((2 * chunk_size + 1) * sizeof(char));

    if (rank == 0) {
        resultant = (char*)malloc((2 * length + 1) * sizeof(char));
    }
}
```

```

    MPI_Scatter(S1, chunk_size, MPI_CHAR, local_s1, chunk_size, MPI_CHAR, 0,
MPI_COMM_WORLD);
    MPI_Scatter(S2, chunk_size, MPI_CHAR, local_s2, chunk_size, MPI_CHAR, 0,
MPI_COMM_WORLD);

    local_s1[chunk_size] = '\0';
    local_s2[chunk_size] = '\0';

    for (int i = 0; i < chunk_size; i++) {
        local_res[2 * i] = local_s1[i];
        local_res[2 * i + 1] = local_s2[i];
    }
    local_res[2 * chunk_size] = '\0';

    MPI_Gather(local_res, 2 * chunk_size, MPI_CHAR, resultant, 2 * chunk_size, MPI_CHAR, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        resultant[2 * length] = '\0';
        printf("Resultant String: %s\n", resultant);

        free(S1);
        free(S2);
        free(resultant);
    }

    free(local_s1);
    free(local_s2);
    free(local_res);

    MPI_Finalize();
    return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 6$ mpicc q2.c -o q2
pcap1@PC-36:~/Documents/240968040/week 6$ mpirun -n 3 q2
Name: Devarjya, Reg No: 240968040
Enter string S1: string
Enter string S2: length
Resultant String: slternigtgh

```

3) Write a MPI program using N processes to find $1! + 2! + \dots + N!$. Use scan.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    long long local_fact = 1;
    long long scan_sum = 0;
    long long total_sum = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        printf("Name: Devarjya, Reg No: 240968040\n");
    }

    int n = rank + 1;

    for (int i = 1; i <= n; i++) {
        local_fact *= i;
    }

    MPI_Scan(&local_fact, &scan_sum, 1, MPI_LONG_LONG, MPI_SUM,
MPI_COMM_WORLD);

    MPI_Reduce(&local_fact, &total_sum, 1, MPI_LONG_LONG, MPI_SUM, 0,
MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Sum of series (1! + 2! + ... + %d!) = %lld\n", size, total_sum);
    }

    MPI_Finalize();
    return 0;
}
```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 6$ mpicc q3.c -o q3
pcap1@PC-36:~/Documents/240968040/week 6$ mpirun -n 3 q3
Name: Devarjya, Reg No: 240968040
Sum of series (1! + 2! + ... + 3!) = 9
pcap1@PC-36:~/Documents/240968040/week 6$ mpirun -n 5 q3
Name: Devarjya, Reg No: 240968040
Sum of series (1! + 2! + ... + 5!) = 153
pcap1@PC-36:~/Documents/240968040/week 6$

```

4) Write a MPI program to read a 3 x 3 matrix. Enter an element to be searched in the root process. Find the number of occurrences of this element in the matrix using three processes

Code:

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int matrix[3][3];
    int local_row[3];
    int search_elem, local_count = 0, total_count = 0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 3) {
        if (rank == 0) {
            printf("Error: This program requires exactly 3 processes (-np 3).\n");
        }
        MPI_Finalize();
        exit(1);
    }

    if (rank == 0) {
        printf("Name: Devarjya, Reg No: 240968040\n");
        printf("Enter elements of 3x3 matrix:\n");
        fflush(stdout);
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                scanf("%d", &matrix[i][j]);
            }
        }
        printf("Enter element to search: ");
        fflush(stdout);
        scanf("%d", &search_elem);
    }
}

```

```

MPI_Bcast(&search_elem, 1, MPI_INT, 0, MPI_COMM_WORLD);

MPI_Scatter(matrix, 3, MPI_INT, local_row, 3, MPI_INT, 0, MPI_COMM_WORLD);

for (int i = 0; i < 3; i++) {
    if (local_row[i] == search_elem) {
        local_count++;
    }
}

MPI_Reduce(&local_count, &total_count, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

if (rank == 0) {
    printf("Total occurrences of %d in the matrix: %d\n", search_elem, total_count);
}

MPI_Finalize();
return 0;
}

```

Output:

```

pcap1@PC-36:~/Documents/240968040/week 6$ mpicc q4.c -o q4
pcap1@PC-36:~/Documents/240968040/week 6$ mpirun -n 3 q4
Name: Devarjya, Reg No: 240968040
Enter elements of 3x3 matrix:
1
1
1
2
2
2
3
3
3
Enter element to search: 3
Total occurrences of 3 in the matrix: 3

```

6) Write a MPI program to read 4 x 4 matrix display the following output using four processes

I/p matrix	1 2 3 4	O/p matrix:	1 2 3 4
	1 2 3 1		2 4 6 5
	1 1 1 1		3 5 7 6
	2 1 2 1		5 6 9 7

Code:

```

#include <stdio.h>
#include <stdlib.h>

```

```

#include <mpi.h>

int main(int argc, char** argv) {
    int rank, size;
    int matrix[4][4];
    int local_row[4];
    int local_out[4];
    int *output_matrix = NULL;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (size != 4) {
        if (rank == 0) {
            printf("Error: This program requires exactly 4 processes (-np 4).\n");
        }
        MPI_Finalize();
        exit(1);
    }

    if (rank == 0) {
        printf("Name: Devarjya, Reg No: 240968040\n");
        printf("Enter elements of 4x4 matrix:\n");
        fflush(stdout);
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                scanf("%d", &matrix[i][j]);
            }
        }
        output_matrix = (int*)malloc(4 * 4 * sizeof(int));
    }

    MPI_Scatter(matrix, 4, MPI_INT, local_row, 4, MPI_INT, 0, MPI_COMM_WORLD);

    MPI_Scan(local_row, local_out, 4, MPI_INT, MPI_SUM, MPI_COMM_WORLD);

    MPI_Gather(local_out, 4, MPI_INT, output_matrix, 4, MPI_INT, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("Output Matrix:\n");
        for (int i = 0; i < 4; i++) {
            for (int j = 0; j < 4; j++) {
                printf("%d ", output_matrix[i * 4 + j]);
            }
            printf("\n");
        }
        free(output_matrix);
    }

    MPI_Finalize();
    return 0;
}

```


}

Output:

```
pcap1@PC-36:~/Documents/240968040/week 6$ mpicc q6.c -o q6
pcap1@PC-36:~/Documents/240968040/week 6$ mpirun -n 4 q6
Name: Devarjya, Reg No: 240968040
Enter elements of 4x4 matrix:
1
2
3
4
1
2
3
1
1
1
1
1
2
1
2
1
Output Matrix:
1 2 3 4
2 4 6 5
3 5 7 6
5 6 9 7
```